

OOPS

21

Practical use

()

needs

→ String

Stack

miss oop

#

class & Objects



Blueprint
of these

entities



How it ~~look~~
look

any real world entity
Item

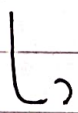
#

Method



function

written inside class



member

function

CLASS

class TEACHER {
PUBLIC:

String name;

String dept;

String subject;

double salary;

} Properties

void changeDept (String newDept) {

dept = newDept;

}

};

int main() {

TEACHER T1;

T1.name = "Kanu";

T1.subject = "C++";

T1.dept = "CSE";

T1.salary = 25500;

cout

}

ACCESS MODIFIERS

- o Private → data & method accessible inside class
- o Public → everyone can access
- o Protected → data & methods accessible inside class & its derived class

SETTER → private value set

```
void setter (double s) {
    salary = s; }
}
```

getter → private value get

```
double getter () {
    return salary; }
}
```

```
"ema" = ema .IT
```

```
"44" = 44 .IT
```

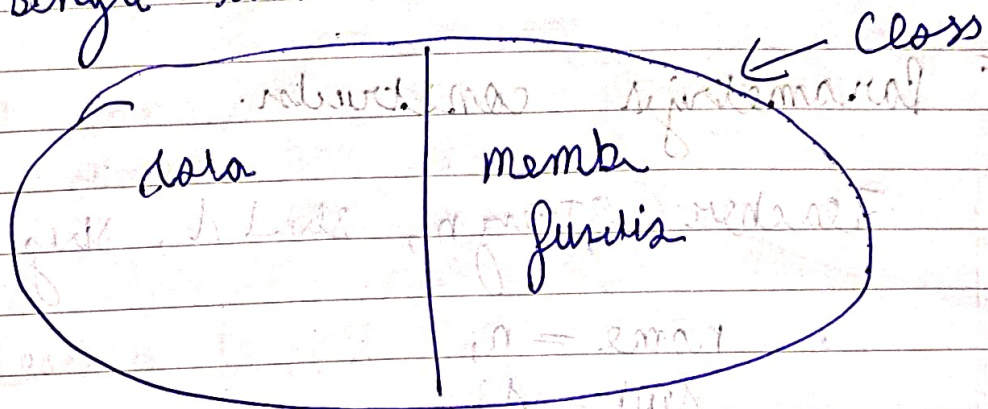
```
"92" = 92 .IT
```

```
"0022" = 0022 .IT
```

```
1700
```

ENCAPSULATION

wrapping up of data & member functions
in a single unit class



o help in data hiding

↳ imp info ko private bna do

constructor — ^{always public} when object create constructor ~~called~~ get call.

default set

`Teacher() {
 dept = CSE
}` // constructor

o To initialise value

o memory allotment happen when it called.

|| Non Parametrized constructor

```
Teacher() {
    dept = "comp";
}
```

Parametrized constructor.

```
Teacher (String n, short d, String s, double sal) {
```

```
    name = n;
```

```
    dept = d;
```

```
    subject = s;
```

```
    salary = sal;
```

```
} // and parameterized
```

```
int main() {
```

```
    Teacher t1("Kanu", "CSE", "JAVA", 25000);
}
```

different Types of parameters

of constructor is - constructor

~~overloading~~ overloading

Polymorphism

This $\rightarrow$ name = name; // use Karo in constructor m.

Copy Constructors

special constructor used to copy properties of one object into another.

Teacher t2(t1)

$\hookrightarrow$ copy constructor

we can also make our own copy constructor

Teacher { Teacher &obj obj }

This $\rightarrow$ name = obj.name;

}

Shallow Copy

↳ default constructor make shallow copy

- Object copy all member value from one obj to another.

* Issue come when we do dynamic mem allocation.

- it work in static allocated mem.
- if next obj modify both object
link

Deep copy → complete independent

Object with all its own copy of next objects

if you change some in new obj

not affect the original one.

- No link

Destructors

✓ class Name() {

...}

• deallocate memory

in C++ if O/B3 → Dynamic mem

(by new keyword)

and delete key for dealloc

main

ipad

if type is ...

main = main + 1

ipad = ipad + 1

Inheritance - code reusability.

Properties and member functions of base class are passed on to the derived class.

Class A (Parent, Base) class



inherit

Class B (Child, Derived)

```
class Person {
```

```
    public:
```

```
        String name;
```

```
        int age;
```

```
        Person (string name, int age) {
```

```
            this.name = name;
```

```
            this.age = age;
```

```
        }
```

o inheritance

```
class student : Public Person {
```

```
    Public roll no;
```

```
    void getinfo() {
```

```
        cout << "name ";
```

```
        cout << age;
```

```
        cout << roll no;
```

```
    };
```

first call

Base class constructor
Derive class constructor

for call

Derive class constructor
~~Base~~ Base class constructor

MODE of inheritance

o Private

o Protected

o Public

Single level

Parent
↓
child

Multi level inheritance

P
↓
P
↓
C

Hierarchical

P
↙ ↘
C C

Hybrid

→ see inheritance
mix

Polymorphism

many

form

ability of object to take on different forms

or behave different ways depend on the

context

o compile time polymorphism →

Ex → function overloading

constructor overloading

operator overloading

o Run Time Polymorphism

- Ex. ① function overriding

Both parent & child have
same ~~param~~ parameter ~~name~~ But
different implementation

① ~~Virtual~~ Virtual function.

↳ member function that you can

expect to be redefined in derived class

could not be overridden

~~Virtual~~ Virtual void hello() {

cout << hi
}

o ~~Virtual~~ Virtual fns are ~~Dynamic~~ Dynamic in nature

o Define the key for Virtual in Base class

as overridden in a derived class.

// Virtual fns called during runtime.

Abstraction

() hiding all unnecessary details showing only the important parts

can be done by

① access modifier / specifier

② Abstract class

Abstract Class {

}

Instance mean = Object

- abstract class is Template it set up structure

But actual work is done by

Child class.

- can't directly create Object

Static

- Static variable - created one time of msk for function

- They share all ^{Object} ~~object~~ of class

Static Objects

Interference $\rightarrow$ a Blueprint that contains any method declaration.

• was ~~100~~ for 1007. obstruction

Interference = method signature

for the π development of any method.

get method of stream output tool

class (kind) tool

method stream output tool

2. Java

is must all things - stream output tool

1. java

can't go to the end with a

output stream